A. Process & การสร้าง Process ด้วย fork / exec / wait (OSNo03)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Process = โปรแกรมที่ "มีชีวิต" กำลังทำงานอยู่ใน RAM มีสถานะและทรัพยากรของตัวเอง · fork = คำสั่งแตกสำเนา process (กระบวนการ), exec = เปลี่ยนโปรแกรมที่ process กำลังรัน, wait = ให้พ่อรอลูกจบ

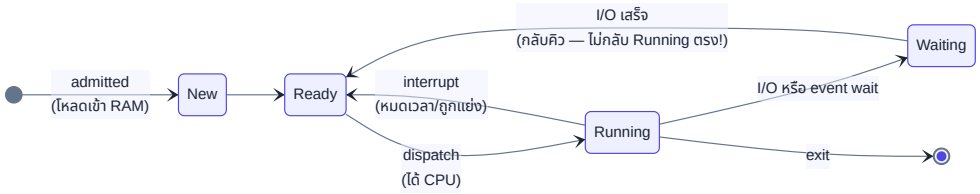
ความสำคัญกับข้อสอบ: ข้อสอบชอบให้ "ไล่โค้ด fork" — นับว่ามี process เกิดกี่ตัว เดาว่า output พิมพ์อะไร รุ่นพี่เตือนว่ามีจุดหลอกเยอะ และย้ำว่าโค้ดทุกข้อ compile ผ่านหมด ไม่มีคำตอบว่า Error

ใช้จริงในคอมพิวเตอร์: ทุกครั้งที่พิมพ์คำสั่งใน terminal หรือดับเบิลคลิกเปิดแอป คือ shell (โปรแกรมรับคำสั่งผู้ใช้) ทำ fork → exec → wait ให้เราทั้งหมด — เช่น พิมพ์ ls, เปิด Chrome, compile ไฟล์

A1. Program vs Process · สถานะของ Process

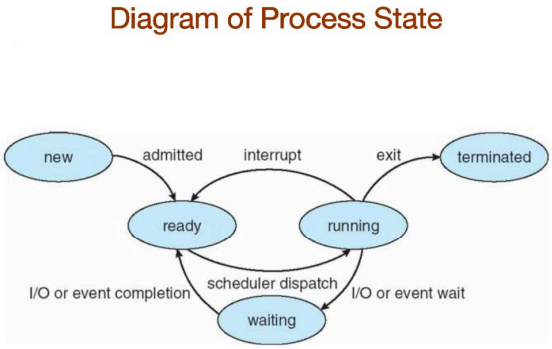
	Program	Process
คือ	ไฟล์คำสั่งนิ่ง ๆ บนดิสก์ — **passive entity**	**program in execution** — active entity ต้องใช้ทรัพยากร
มีอะไร	แค่โค้ด	Program counter (PC), stack, data (ข้อมูล) section, heap, registers
ชื่อเรียกอื่น	—	Task (time-sharing) · Job (batch system)

สถานะของ process (state diagram):

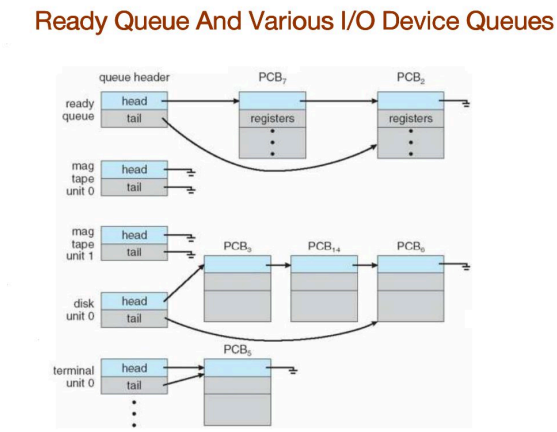


🎯 จุดหลอก

Waiting เสร็จแล้วไป **Ready ก่อนเสมอ** ไม่ใช่กลับ Running ทันที (ต้องรอ dispatcher เรียก) · Batch system เรียก process ว่า Job, time-sharing เรียก Task



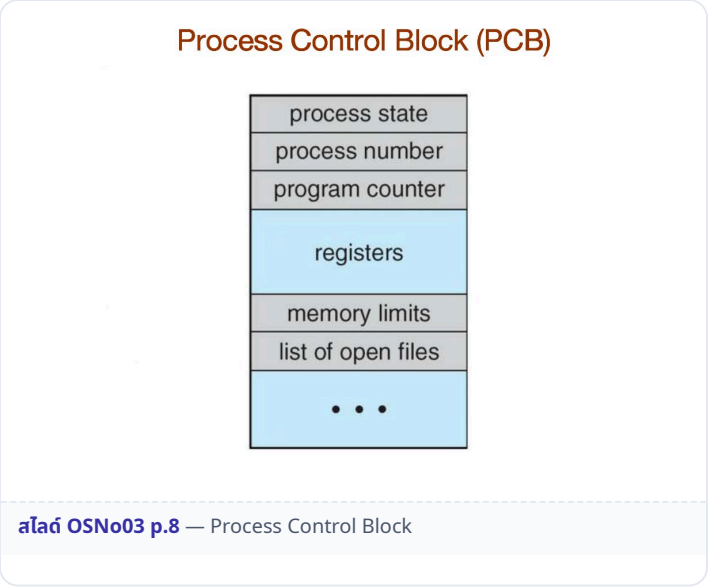
สไลด์ OSNo03 p.4 — Diagram of Process State (ต้นฉบับอาจารย์)



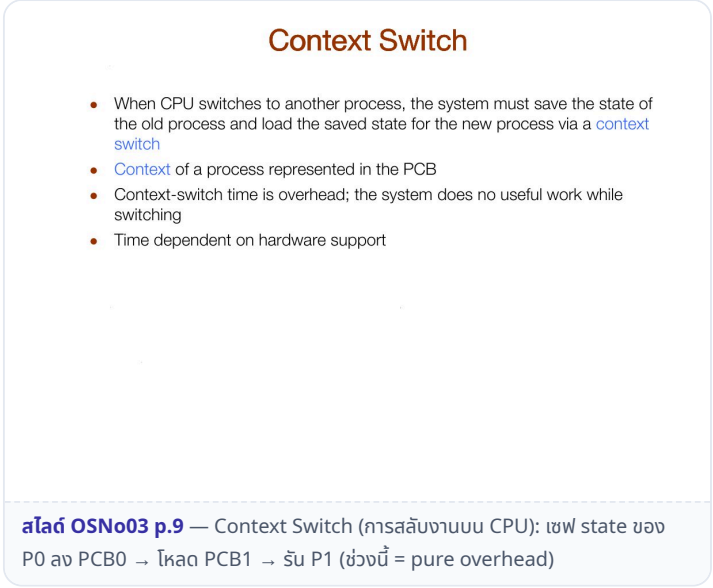
สไลด์ OSNo03 p.7 — Ready Queue (คิวงานที่พร้อมรัน) และ I/O Device Queues: process วิ่งวน Ready → CPU → I/O คิว

A2. PCB — Process Control Block (บล็อก (หน่วยจัดเก็บ))

PCB = "รหัสประชาชน" ของ process เก็บทุกอย่างที่ OS ต้องรู้: **process state, program counter, CPU registers, scheduling (การจัดคิว) info (priority (ลำดับสิทธิ์)/queue), memory (หน่วยความจำ) info (base/limit), accounting info, I/O status**



สไลด์ OSNo03 p.8 — Process Control Block



สไลด์ OSNo03 p.9 — Context Switch (การสลับงานบน CPU): เซฟ state ของ P0 ลง PCB0 → โหลด PCB1 → รัน P1 (ช่วงนี้ = pure overhead)

Dispatcher (ผู้ส่งงานเข้า CPU) ทำ 3 อย่าง

(1) switching context (2) switch to user mode (3) jump ไปตำแหน่งเดิมใน user program · **Dispatch latency** = เวลาที่ dispatcher ใช้หยุดตัวหนึ่ง เริ่มอีกตัวหนึ่ง

A3. fork() — สร้าง process แบบโคลนตัวเอง

`pid_t fork(void)` สร้าง child โดย**ก๊อปปี้ parent** (address (ที่อยู่หน่วยความจำ) space, ตัวแปร, fd ทุกอย่าง) แล้วทั้งคู่รันต่อจากบรรทัดเดียวกัน (หลัง fork)

ใครเรียก	ค่าที่ fork() คืน	เงื่อนไข if
ใน Parent	**PID ของลูก** (> 0)	`if (pid > 0)`
ใน Child	**0**	`else if (pid == 0)`
สร้างไม่สำเร็จ	**ค่าลบ**	`if (pid < 0)` → error

🪄 เข้าใจง่าย

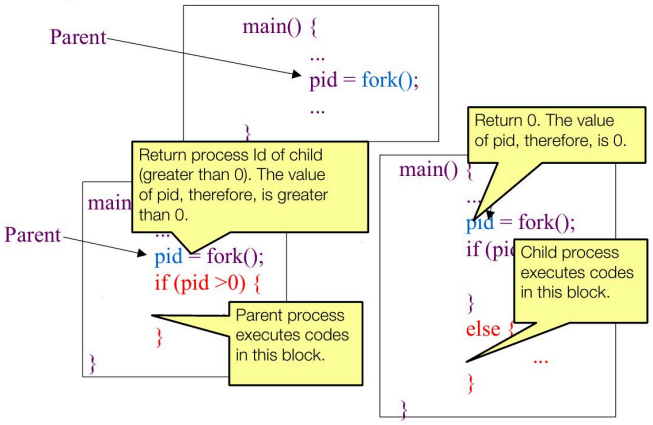
fork() เรียก**ครั้งเดียวแต่คืนค่า 2 ครั้ง** — ครั้งนึงใน "โลกพ่อ" ครั้งนึงใน "โลกลูก" ทั้งคู่ถือ copy ของตัวแปร pid ของตัวเอง แก้ฝั่งไหนไม่กระทบอีกฝั่ง

```c
/* fork_example.c — ตัวอย่างจากสไลด์/วิดีโอ: ผลลัพธ์พิมพ์ 2 บรรทัด */
#include <stdio.h>
#include <sys/types.h>   // pid_t
#include <unistd.h>      // fork()

int main(void) {
    pid_t pid;                  // เก็บค่า return ของ fork
    int i = 0;                  // ตัวแปรธรรมดา — จะถูก "ก๊อป" ไปให้ลูกด้วย

    pid = fork();               // ← จุดแตกหน่อ: หลังบรรทัดนี้มี 2 process รันโค้ดชุดเดียวกัน
    if (pid > 0) {              // โลกพ่อ: ได้ PID ของลูก (>0) เสมอ
        i = 1;                  // แก้ i ใน address space ของพ่อ — ลูกไม่เห็น
        printf("I am parent i = %d\n", i);
    }
    else {                      // โลกลูก: fork คืน 0
        i = 2;
        printf("I am child i = %d\n", i);
    }
    return 0;
}
/* Output: "I am parent i = 1" และ "I am child i = 2" (2 บรรทัดเสมอ)
   แต่ลำดับก่อน-หลัง non-deterministic — ขึ้นกับ scheduler ห้ามสรุปใครพิมพ์ก่อน */
```

Diagram for fork() example



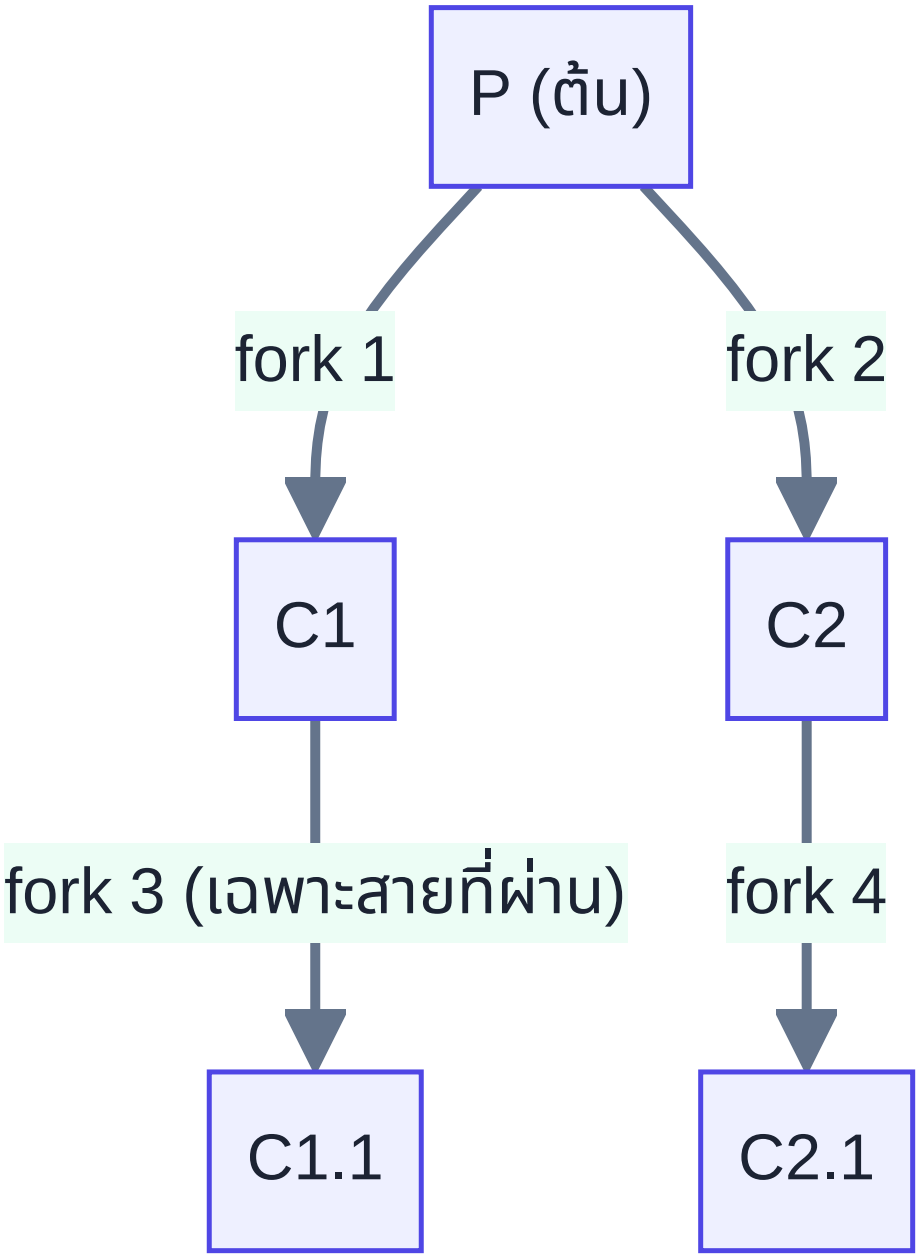
สไลด์ OSNo03 p.19 — Diagram for fork() example: พ่อ/ลูกรันต่อจากจุดเดียวกัน แยกคนละสาย

Process IDs (pids)

- pid = fork();
- In the child: pid == 0;
  In the parent: pid == the process ID of the child.
- If the value of pid is negative, it indicates that the creation of a child process was unsuccessful.
- The application can utilize this difference in the value of pid to execute distinct actions in the parent and child processes.

สไลด์ OSNo03 p.20 — Process IDs: ลูกได้ 0 / พ่อได้ PID ลูก / ติดลบ = fail

A3.1 นับจำนวน process — วาด Process Tree (แผนภาพต้นไม้ความสัมพันธ์พ่อ-ลูก)



`fork()` ติดกัน `n` ครั้ง ไม่มีเงื่อนไข → process ทั้งหมด = 2^n (ลูกใหม่ = $2^n - 1$)

fork 3 ครั้ง = 8 process · ถ้ามี if/exit ให้วาด tree แล้วนับ "สายที่ผ่าน fork จุดนั้น" ทีละจุด

A3.2 fork + loop จากวิดีโอ (จุดหลอก: ลูกต้อง exit)

```
for (i = 0; i < 5; i++) {          // พ่อวน 5 รอบ
    pid = fork();
    if (pid == 0) {                 // ลูกเท่านั้นเข้า if
        if (i % 2 != 0) printf("I am happy\n");  // ลูกที่เกิดตอน i คี่
        else            printf("I am sad\n");    // ลูกที่เกิดตอน i คู่
        exit(0);                    // ← ถ้าลืม! ลูกจะหลุด if ไปวน for ต่อ
    }                               //   แล้ว fork ลูก/หลานของตัวเองเพิ่ม
}
while (wait(NULL) != -1);           // พ่อรอลูกทุกตัว (wait คืน -1 เมื่อไม่มีลูกเหลือ)
printf("I am parent goodbye\n");
```

🎯 จุดหลอก (อาจารย์สาธิตในวิดีโอ)

มี exit() → sad ×3 (i=0,2,4), happy ×2 (i=1,3), goodbye **1 ครั้งสุดท้าย** · ถ้าลูกเลขคี่**ลืม exit()** → ลูก C1, C3 กลายเป็น "พ่อ" สร้างหลาน → goodbye พิมพ์ **4 ครั้ง** (พ่อหลัก + C1 + C3 + หลาน $C1_{-3}$)

A4. exec() — เปลี่ยนร่าง ไม่สร้างคนใหม่

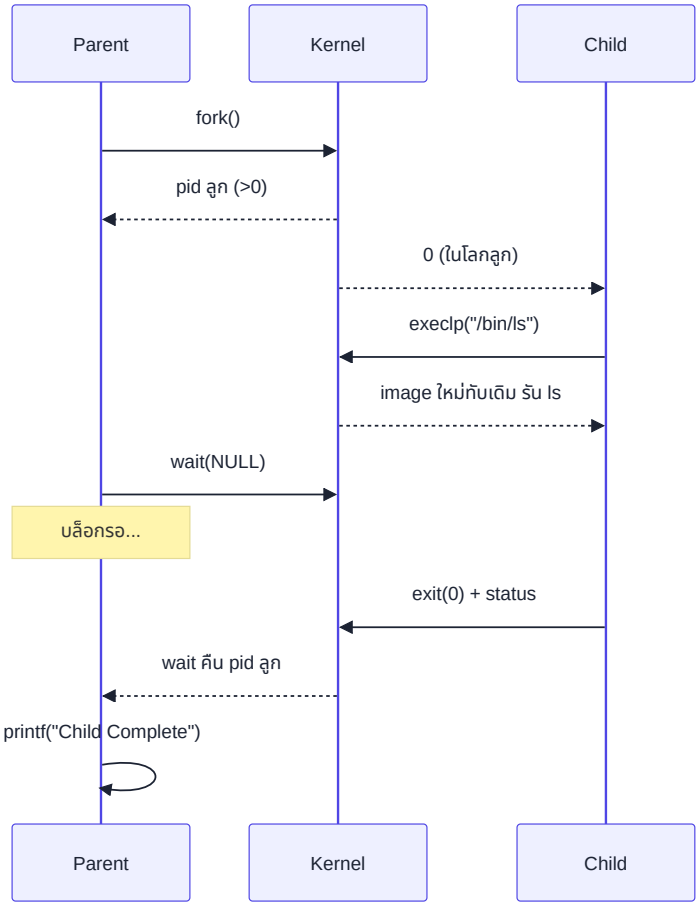
- ตระกูล exec แทนที่ **process image ทั้งหมด** (โค้ด+data) ด้วยโปรแกรมใหม่ — PID คงเดิม
- `execlp(file, arg0, arg1, ..., NULL)` — **l** = list (ส่งทีละตัว) · **p** = หาโปรแกรมตาม PATH · **arg0** = ธรรมเนียมใส่ชื่อโปรแกรม (exec ข้าม แต่ต้องใส่) · **NULL ปิดท้ายบังคับ**
- execv: ส่งเป็น array `char *argv[]`

🎯 จุดหลอก exec

exec **สำเร็จ** → โค้ดหลังบรรทัด exec **ไม่มีวันรัน** (image โดนทับหมด) · exec **ล้มเหลว** (ชื่อผิด เช่น "ly") → รันบรรทัดถัดไปต่อ · `exec("cd")` fail เสมอ เพราะ **cd เป็น shell built-in** ไม่ใช่ไฟล์ (echo ได้เพราะมี /usr/bin/echo)

A5. fork + exec + wait — รูปแบบมาตรฐานของ shell

```c
int main() {
    pid_t pid = fork();
    if (pid < 0) {                          // สร้างลูกไม่สำเร็จ
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) {                    // ── ลูก: เปลี่ยนร่างเป็น ls ──
        execlp("/bin/ls", "ls", NULL);      // สำเร็จ = โค้ดลูกถูกแทนที่ทั้งหมด
    }
    else {                                  // ── พ่อ: รอลูก ──
        wait(NULL);                          // บล็อกรอลูกตัวใดตัวหนึ่งจบ (ได้ pid ลูกกลับ)
        printf("Child Complete");           // พิมพ์หลังลูกจบเสมอ
        exit(0);
    }
}
/* Output: ผล ls ก่อน ตามด้วย "Child Complete" — ลำดับแน่นอนเพราะ wait() */
```



A6. exit / wait / waitpid (รอลูกที่ระบุตัวเจาะจง) + สถานะพิเศษ

Call	ทำอะไร	จุดสำคัญ
`exit(status)`	ขอ OS ทำลายตัวเอง ส่ง status ให้พ่อ	ส่งได้แค่ **8 บิต (0–255)** — exit(300) พ่ออ่านได้ 300 mod 256 = 44
`wait(&status)`	รอ*ลูกใดก็ได้*จบ	คืน pid ลูกที่จบ · ไม่มีลูก → คืน **-1** · ไม่สน status ใช้ wait(NULL)
`waitpid(pid, &status, 0)`	รอ*ลูกตัวเจาะจง*	พ่อรายงานผลเรียงลำดับได้ (ต่างจาก wait ที่ได้ใครมาก่อน)
`WIFEXITED(status)`	เช็คลูกจบแบบปกติ?	พิมพ์ status ตรง ๆ ได้เลขแปลก (32768 ฯลฯ) เพราะ OS แปะข้อมูลอื่นปน — **ต้อง decode ด้วยแมโคร**
`WEXITSTATUS(status)`	ดึงค่า exit จริง	

🧟 Zombie (กระบวนการที่ตายแล้วแต่รายการยังค้างในระบบ) process

ลูก**ตายแล้ว**แต่พ่อไม่ wait() → ทรัพยากรถูกคืนแต่รายการใน process table (ตาราง) ค้าง สถานะ **Z / defunct** — ทางแก้: พ่อต้อง wait

👶 Orphan (กระบวนการที่พ่อตายก่อน) process

พ่อ**ตายก่อนลูก** → OS ให้ init/systemd "รับเลี้ยง" (ps -l เห็น PPID เปลี่ยน) · **Cascading termination**: พ่อโดน terminate แล้ว OS บางระบบลบลูกหลานตามทั้งตระกูล

A6.1 ประสานพ่อ-ลูก: wait vs waitpid (จากวิดีโอ parent.c/child.c)

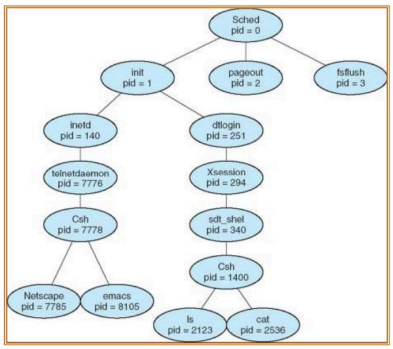
```c
/* child.c: รับ pid ตัวเอง สุ่ม sleep 0-4 วิ แล้ว exit(pid % 256)
   parent.c: fork 3 ลูก (pid[i]) แล้วเปรียบเทียบสองแบบ */
// แบบ 1: wait(&status) → ลูกตัวไหนจบก่อน พ่อรายงานตัวนั้นก่อน (ลำดับไม่แน่)
while ((pid = wait(&status)) > 0) printf("child %d done\n", pid);
// แบบ 2: waitpid(pid[i], &status, 0) → พ่อรอ "ตามลำดับที่กำหนด" เสมอ
for (i = 0; i < 3; i++) {
    waitpid(pid[i], &status, 0);   // นิ่งรอลูกตัวที่ 1 ก่อน แม้ตัว 3 จบก่อน
    printf("child %d done\n", pid[i]);
}
```

Fork and exec()

```c
int main()
{
    pid_t  pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
            fprintf(stderr, "Fork Failed");
            exit(-1);
    }
    else if (pid == 0) { /* child process */
            execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
            /* parent will wait for the child to complete */
            wait (NULL);
            printf ("Child Complete");
            exit(0);
    }
}
```

สไลด์ OSNo03 p.23 — Fork and exec(): โครงโค้ดมาตรฐานจากสไลด์อาจารย์

A tree of processes on a typical Solaris



สไลด์ OSNo03 p.15 — Process tree (login → sh → คำสั่งต่าง ๆ)